

An incremental approach to automatic firewall configuration

Candidate: Filippo Del Ministro

Supervisors: prof. Fulvio Valenza, prof. Daniele Brighenti, prof. Francesco Pizzato

1 Introduction

Modern enterprise networks are large and highly dynamic infrastructures: new services are deployed, nodes are added or removed, and connectivity requirements evolve continuously. Enforcing security under these conditions is an ongoing operational task, and packet-filtering firewalls remain the primary mechanism for doing so. In this context, manual configuration represents a risky solution: as networks grow, the volume of rules to manage quickly exceeds human capacity, and manual processes offer no guarantee of correctness, so a single misconfiguration can silently violate a security requirement or block legitimate traffic.

VEREFOO was developed to address this gap through full automation: given a network topology and a set of Network Security Requirements (NSRs), it automatically determines firewall placement and filtering rules, formally guaranteeing correctness and optimality by construction. However, VEREFOO follows a monolithic strategy: any change to the NSR set triggers a complete recomputation from scratch, which is poorly suited to environments where requirements change frequently.

React-VEREFOO addresses this limitation with an incremental approach, reconfiguring only the portion of the network affected by the new requirements instead of recomputing the whole configuration from scratch (Section 2). This results in significant speedups when only a small fraction of the requirements changes, but performance converges toward the monolithic baseline as the fraction of new requirements grows.

This thesis proposes to push the incremental principle to its extreme. Rather than presenting React-VEREFOO with a batch of new requirements, the proposed iterative approach performs a re-configuration by introducing new NSRs one at a time, invoking React-VEREFOO internally at each step to compute the minimal update to the current configuration. By construction, the fraction of new requirements at each step is always minimal, so the performance advantage of the incremental approach is consistently in effect, regardless of how many requirements must ultimately be incorporated.

2 Background

Network security architectures have evolved alongside this growing complexity. Traditionally, a single perimeter firewall placed at the network bound-

ary was sufficient to control traffic ingress and egress traffic of the network, but this model cannot enforce fine-grained policies across differentiated internal topologies. Modern networks have therefore moved toward microsegmentation, deploying multiple packet-filtering firewall instances distributed across the topology (Figures 1 and 2). This detailed control comes at a significant cost of higher management complexity: each firewall instance must be configured individually, and all rules must collectively enforce the intended policy without conflicts or gaps.

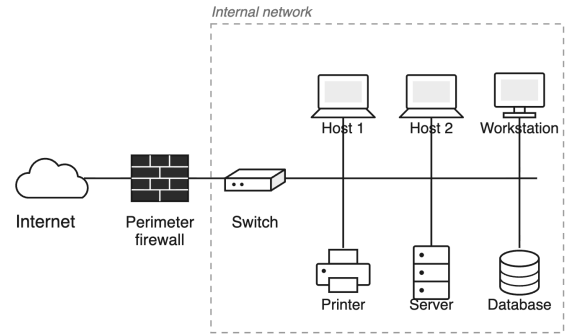


Figure 1: Traditional perimeter firewall. A single point of control at the network boundary; internal traffic is unfiltered.

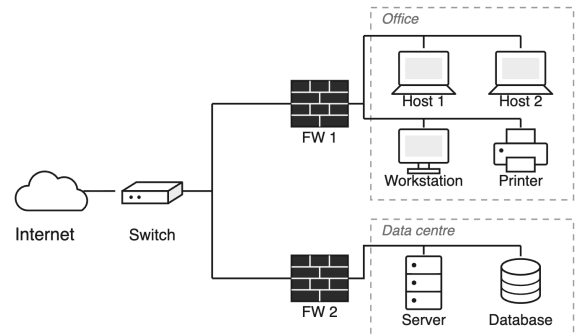


Figure 2: Distributed firewall architecture (microsegmentation). A filtering instance guards each segment.

VEREFOO addresses exactly this configuration problem. Given a network topology, represented internally as a Service Graph, and a set of NSRs describing the requirements of the network — mean-

ing, which traffic flows must be allowed or denied — VEREFOO finds a proper allocation scheme and firewall configuration to enforce the requirements. To do so, VEREFOO derives an Allocation Graph in which every link of the network is annotated as a candidate firewall position (called an Allocation Place) and encodes the placement and configuration problem as a MaxSMT instance. Solving it generates a firewall allocation scheme and a filtering policy for each allocated instance, minimising the number of firewalls and rules while guaranteeing that every NSR is satisfied by construction.

React-VEREFOO builds on VEREFOO to support reconfiguration of nodes, while novel requirements are presented to the same already-configured topology. Given an existing configuration and a new target set of NSRs, React-VEREFOO classifies every requirement as added, kept, or deleted, identifies which nodes are in conflict with the added requirements, and restricts the MaxSMT problem to only those nodes, biasing the solver toward reusing the existing allocation and rules wherever possible. This substantially reduces solving time, and the benefit is greatest when only a small fraction of the requirements changes. As that additional requirement grows, however, the advantage are reduced. This final observation motivates the iterative approach developed in this thesis.

3 Thesis Objectives

The key performance parameter of React-VEREFOO is the proportion of new requirements introduced at once: the higher it is, the wider the network area that must be reconsidered, and the smaller the resulting speedup. The idea developed in this thesis is to push this observation to its logical extreme: instead of processing all new NSRs together, the iterative approach introduces them one at a time. At each step, exactly one new requirement is presented to React-VEREFOO as added, while everything enforced so far is treated as kept; the resulting allocation and rules become the starting point for the next step (Figure 3). The proportion of additional requirements is therefore always at its theoretical minimum, regardless of how many requirements must ultimately be incorporated, so the reconfiguration area identified at each step is always the smallest one possible.

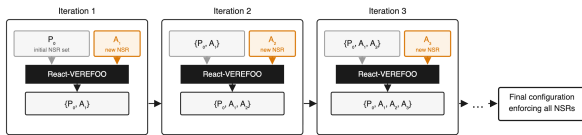


Figure 3: The iterative approach: each step introduces exactly one new NSR, so the solver always sees the smallest possible incremental problem.

This design is expected to yield consistent performance improvements over both vanilla VEREFOO and standard React-VEREFOO, independently of the size of the requirement batch. The main trade-off is the linear number of solver invocations, one per new requirement, each carrying a fixed overhead for parsing, encoding, and solver initialisation that accumulates across iterations. A second trade-off concerns configuration quality: because each step only optimises locally, a firewall placed early to satisfy one requirement may not be the best structural choice once later requirements are taken into account, unlike a monolithic solve which optimises globally over the full requirement set.

4 The Iterative VEREFOO Approach

The iterative approach builds on React-VEREFOO as an internal primitive, invoking it multiple time with exactly one new requirement at a time instead of a batch of new requirements. The algorithm maintains two evolving structures across iterations: **consideredProps**, the set of NSRs already enforced and passed as the *initial* set to React-VEREFOO and **currentGraph**, the network graph as updated by the most recent SAT result. Both are initialised before the loop begins: **consideredProps** with any pre-existing NSRs, and **currentGraph** with the original input graph. Every iteration thus receives a minimal incremental problem, with only one new requirement on top of a stable, already-verified configuration.

Table 1 summarises how the three variants compare.

Table 1: Structural comparison of the three VEREFOO variants.

	Vanilla	React	Iterative
Solver calls	1	1	one per NSR
Added NSRs/call	all at once	batch	always 1
Reuses config	no	yes	yes (each step)
Optimality	global	local	local (greedy)

(A) ITERATIVE SOLUTION

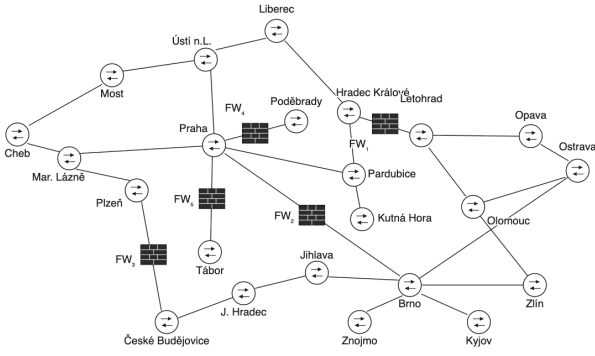


Figure 4: Iterative firewall configuration over the full 15-NSR CESNET set.

(B) MONOLITHIC SOLUTION

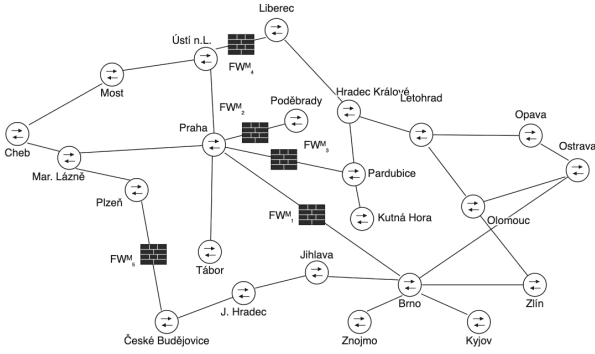


Figure 5: Monolithic firewall configuration over the same 15-NSR CESNET set: three placements coincide with the iterative one, two differ.

5 Implementation and Validation

The iterative variant was implemented by modifying `VerefooSerializer`, the entry point shared by all three algorithm variants: rather than passing the full batch of new requirements to `React-VEREFOO` in a single call, the constructor wraps it in a loop that introduces one NSR at a time, updating `consideredProps` and `currentGraph` between iterations. All downstream components — the `MaxSMT` encoding, `VerefooProxy` and the `Translator` — are left unchanged.

The approach was validated qualitatively on CESNET, the Czech academic backbone (23 nodes, 28 links). Starting from an initial configuration of ten NSRs enforced by three firewalls, five new requirements were introduced one at a time; every iteration was traced step by step to confirm that the new requirement was satisfied without breaking any of the previous ones, ending with five firewalls enforcing all fifteen requirements.

For comparison, vanilla `VEREFOO` was also run once, monolithically, over the same fifteen requirements. Both approaches allocate exactly five firewalls, three of them at identical positions, but the iterative configuration needs one additional rule: the monolithic solver sees the full requirement set at once and can let a single firewall serve two purposes, whereas the iterative process must commit to each placement before later requirements are known (Figures 4 and 5).

6 Conclusion and Future Work

Manual firewall configuration does not scale with the size and complexity of modern networks, which motivated `VEREFOO` as a fully automated solution; because modern networks change frequently, however, `VEREFOO`'s monolithic strategy scales poorly, motivating `React-VEREFOO`'s incremental reconfiguration. This thesis pushed that same principle to its extreme, introducing exactly one NSR at a time so that every individual subproblem stays as small as possible, at the cost of turning one solver call into a chain of many smaller ones.

Validated qualitatively on the CESNET topology, the iterative approach reached a configuration that matches the monolithic baseline in firewall count and in rule configuration, despite never having visibility beyond the single requirement being processed at each step. The comparison between the two versions is still useful despite the tie in firewall count. Unlike a monolithic re-run, the iterative process never relocates an already-valid firewall once a new requirement is introduced, so it avoids the operational cost and the transient insecure states of such relocations, in line with how requirements actually accumulate over time in real networks. Additionally, scalability is expected to be the main advantage of the iterative version: resolving the whole configuration from scratch should grow increasingly expensive relative to solving one requirement at a time as topologies grow larger and reconfigurations become more frequent and substantial.

Future work should prioritise a quantitative evaluation of execution time across networks and NSR sets of varying scale, to establish whether the iterative approach is actually faster than the monolithic baseline. Two further directions could improve the approach itself: a traffic-aware ordering of NSRs to reduce the reconfigurations caused by ties between equally-cheap candidates, and support for NSR removal alongside the additions handled in this thesis.